Lecture 39: Deep Learning (Part-4) — Practice Problem Solutions (Hinglish)

Sabhi 15 practice problems ke complete step-by-step mathematical solutions aur explanations.

Solution 1: Deep Learning Taxonomy aur Inductive Biases

- **FNN / MLP**: Tabular data ke liye bana hai jahan features ke beech koi spatial (aas-paas ka) relation nahi hota. Poore matrix multiplication se har feature har neuron se connect hota hai.
- **CNN**: Images ke liye bana hai do mukhya niyam use karke:
    1. *Local Receptive Field*: Har neuron sirf padosi pixels ke chote patch ko dekhta hai.
    2. *Weight Sharing*: Ek hi filter poori photo par ghoomta hai, jisse parameters 99% kam ho jate hain aur translation invariance milti hai.
- **RNN / LSTM**: Time-series aur sequential data ke liye bana hai jahan pichla word agle word ko influence karta hai ($\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$).

Solution 2: Digital Image Matrix Representation

1. **Dimensions**:
    - Channels-Last (H, W, C): $(1080, 1920, 3)$
    - Channels-First (C, H, W): $(3, 1080, 1920)$
2. **Total Numbers (Pixels)**:

$$1080 \times 1920 \times 3 = \mathbf{6,220,800} \text{ numbers}$$

3. **RGB Values**:
    - Pure Yellow (Red + Green): $(255, 255, 0)$
    - Pure Cyan (Green + Blue): $(0, 255, 255)$
    - 50% Medium Gray: $(128, 128, 128)$

Solution 3: Pixel Normalization Ki Zaroorat

1. **Gradient Explosion Se Bachav**: Agar pixels 255 tak bade honge, toh activations bohot bade ho jayenge aur sigmoid/tanh jaise functions saturate hokar zero gradient dene lagenge (Vanishing Gradient).
2. **Symmetrical Loss Contour**: Normalization ($x/255.0$) sabhi features ko $[0.0, 1.0]$ ke beech la deta hai, jisse loss function ek symmetrical katora ban jata hai aur standard learning rate ($\eta = 0.001$) se gradient descent tezi se minimum tak pahunchta hai.

Solution 4: Computer Vision Ke 4 Mukhya Tasks

Task	Sawaal	Output	Popular Model
1. Classification	Tasveer kiski hai?	Single label (e.g. 'Cat')	ResNet, VGG
2. Class + Loc	Object kahan hai?	Label + 1 Bounding Box (x, y, w, h)	Custom CNN
3. Object Detection	Sabhi objects kahan hain?	Multiple boxes aur labels	YOLO, Faster R-CNN
4. Semantic Segmentation	Har pixel kis class ka hai?	2D Pixel Class Mask	U-Net, DeepLab

---

Solution 5: Autonomous Vehicles Mein Multi-Task Vision

Self-driving cars ek hi backbone CNN network use karti hain jiske multiple heads hote hain:

- Ek head sadak aur gaddhon (potholes) ko pixel-level par segment karta hai.
- Dusra head aas-paas ki gaadiyon aur logon ke 3D bounding boxes nikalta hai.
- Teesra head traffic lights aur speed limit signs ko classify karta hai. Yeh sab real-time mein 30+ frames per second par execute hota hai.

Solution 6: FNN Mein Parameter Explosion Ka Ganit

1. **Parameters Calculation**:
    - Input pixels = $1920 \times 1080 \times 3 = 6,220,800$
    - Neurons = 256

$$\text{Weights} = 6,220,800 \times 256 = \mathbf{1,592,524,800}$$

$$\text{Biases} = 256$$

$$\text{Total Parameters} = \mathbf{1,592,525,056} \text{ parameters!}$$

2. **Kyun Fail Hoga?**:
    - Sirf pehli layer ke weights store karne ke liye > 6.3 GB RAM chahiye hogi (aur Adam optimizer ke sath > 19 GB).
    - 1.5 Billion parameters training photos ko memorize kar lenge aur nayi photos par 0% accuracy denge (Overfitting).

Solution 7: Spatial Locality aur Translation Invariance

- **Spatial Locality**: Tasveer mein paas-paas ke pixels aapas mein milkar aankh, naak ya line banate hain. Flatten karne se padosi pixels ek dusre se door chale jate hain aur 2D geometry nasht ho jati hai.
- **Translation Invariance**: Billi photo mein chahe left mein ho ya right mein, feature wahi rehta hai. CNN mein ek hi filter poori photo par slide karta hai, isiliye billi jahan bhi hogi, filter usko turant detect kar lega.

Solution 8: CNN Ka Do-Stage Blueprint

```
[ Input Photo ]
       |
       v
[ Stage 1: Feature Extraction ] (Conv2d -> ReLU -> MaxPool2d)
       |
       v  <--- 3D Tensor of Feature Maps (Channels x Height x Width)
[ Stage 2: Classification Head ] (Flatten -> Linear -> Dropout -> Softmax)
       |
       v
[ Output Category Prediction ]
```

Stage 1 photo se important visual features extract karta hai (3D tensor), aur Stage 2 usko flatten karke final category predict karta hai.

Solution 9: Hierarchical Visual Feature Learning

1. **Shuruati Layers (Low-Level)**: Chote 3×3 filters seedhe pixels ko dekhkar horizontal lines, vertical lines aur edges detect karte hain.
2. **Beech Ki Layers (Mid-Level)**: In lines ko jodkar curves, corners, circles aur textures banaye jate hain.

3. **Deep Layers (High-Level)**: Textures ko combine karke car ke pahiye, billi ke kaan aur chehre detect hote hain.

Solution 10: 2D Convolution Ka Manual Calculation

Given:

$$X = \begin{bmatrix} 2 & 0 & 1 & 3 \\ 1 & 2 & 0 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 1 & 0 & 2 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}, \quad b = 1, \quad S = 1, \quad P = 0$$

1. **Output Size**:

$$O = \frac{4 - 2 + 0}{1} + 1 = \mathbf{3} \implies \mathbf{3 \times 3}$$

2. **Top-Left Cell** $(0, 0)$:

$$\begin{aligned} y_{0,0} &= (2 \times 1) + (0 \times -1) + (1 \times 2) + (2 \times 0) + 1 \\ &= 2 + 0 + 2 + 0 + 1 = \mathbf{5} \end{aligned}$$

3. **Top-Right Cell** $(0, 2)$:

$$\begin{aligned} y_{0,2} &= (1 \times 1) + (3 \times -1) + (0 \times 2) + (2 \times 0) + 1 \\ &= 1 - 3 + 0 + 0 + 1 = \mathbf{-1} \end{aligned}$$

Solution 11: Handcrafted Sobel Filters vs Learnable CNN Kernels

- **Sobel Filters**: Purane zamane mein researchers ne pen-paper se numbers fix kiye the (jaise $[-1, 0, 1]$) sirf lines dhoondhne ke liye. Yeh fixed hote hain aur kabhi change nahi hote.
- **CNN Kernels**: Inke numbers shuruat mein random hote hain. Training ke dauran Backpropagation aur Gradient Descent in numbers ko automatically optimize karte hain taaki model dataset ke mutabiq best features seekh sake.

Solution 12: Output Dimension Ka Universal Formula

Formula:

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

1. $W = 32, K = 5, P = 2, S = 1$:

$$O = \frac{32 - 5 + 4}{1} + 1 = 31 + 1 = \mathbf{32 \times 32}$$

2. $W = 224, K = 7, P = 3, S = 2$:

$$O = \left\lfloor \frac{224 - 7 + 6}{2} \right\rfloor + 1 = \left\lfloor \frac{223}{2} \right\rfloor + 1 = 111 + 1 = \mathbf{112 \times 112}$$

3. **Same Padding Ka Proof**: Output size $O = W$ banane ke liye (stride $S = 1$ par):

$$W = W - K + 2P + 1 \implies 2P = K - 1 \implies P = \frac{K - 1}{2}$$

Kyunki K hamesha odd number hota hai $(3, 5, 7)$, isiliye P hamesha ek whole number $(1, 2, 3)$ nikalta hai jo dimensions ko barabar rakhta hai.

Solution 13: Pooling Layer Ke Niyam

1. 2×2 **Max Pooling Calculation**:
   - Top-Left $[4, 9, 8, 2] \implies \max = \mathbf{9}$
   - Top-Right $[1, 3, 7, 5] \implies \max = \mathbf{7}$
   - Bottom-Left $[3, 1, 6, 0] \implies \max = \mathbf{6}$
   - Bottom-Right $[14, 10, 2, 12] \implies \max = \mathbf{14}$

$$\text{Output} = \begin{bmatrix} \mathbf{9} & \mathbf{7} \\ \mathbf{6} & \mathbf{14} \end{bmatrix}$$

2. **Parameters ZERO Kyun?**: Max pooling sirf 4 numbers mein se sabse bada number chunti hai. Isme koi weight matrix (W) ya bias (b) nahi hota. Isiliye $\text{Params} = 0$.
3. **Translation Invariance**: Agar koi feature block ke andar thoda sa hil bhi jaye, tab bhi block ka maximum number wahi rehta hai, jisse output stable rehta hai.

Solution 14: Flattening, Dense Reasoning aur Dropout

1. **Flattened Length**:

$$128 \times 7 \times 7 = \mathbf{6,272} \text{ nodes}$$

2. **Global Reasoning**: Conv layers chote local patches ko dekhti hain. Dense layer saare visual features ko ek sath combine karke poori photo ka global decision leti hai.
3. **Dropout ($p = 0.5$)**: 50% neurons ko randomly band karne se neurons aapas mein interdependent hone ke bajaye independently strong features seekhte hain, jisse overfitting rukti hai.

Solution 15: Poore CNN Network Ka Parameter Calculation

1. **Output Shapes**:
   - Layer 1 (`Conv2d` $K = 3, P = 1, S = 1$): $(\mathbf{32}, \mathbf{32}, \mathbf{32})$
   - Layer 2 (`MaxPool2d` 2×2): $(\mathbf{32}, \mathbf{16}, \mathbf{16})$
   - Layer 3 (`Conv2d` $K = 3, P = 1, S = 1$): $(\mathbf{64}, \mathbf{16}, \mathbf{16})$
   - Layer 4 (`MaxPool2d` 2×2): $(\mathbf{64}, \mathbf{8}, \mathbf{8})$
   - Layer 5 (`Flatten`): $64 \times 8 \times 8 = \mathbf{4,096}$ nodes
   - Layer 6 (`Linear`): $(\mathbf{128})$
   - Layer 7 (`Linear`): $(\mathbf{10})$
2. **Learnable Parameters**:
   - Layer 1: $(3 \times 3 \times 3 + 1) \times 32 = (27 + 1) \times 32 = \mathbf{896}$
   - Layer 2: $\mathbf{0}$
   - Layer 3: $(32 \times 3 \times 3 + 1) \times 64 = (288 + 1) \times 64 = \mathbf{18,496}$
   - Layer 4: $\mathbf{0}$
   - Layer 5: $\mathbf{0}$
   - Layer 6: $4096 \times 128 + 128 = \mathbf{524,416}$
   - Layer 7: $128 \times 10 + 10 = \mathbf{1,290}$
3. **Total Parameters**:

$$\text{Total} = 896 + 18,496 + 524,416 + 1,290 = \mathbf{545,098} \text{ Parameters}$$

(Notice: Akele Layer 6 mein 96.2% parameters shamil hain!)